GOVERNMENT POLYTECHNIC FOR GIRLS, SURAT

[252] COMPUTER ENGINEERING DEPARTMENT
Subject: Introduction to Machine Learning (DI04000061)
Sem – IV | Div.: A/B/C

COMPLETE ASSIGNMENT SOLUTIONS

Assignment 1 • Assignment 2 • Assignment 3

Assignment	Topics Covered	Questions
Assignment 1	Basic ML Concepts, Types, Applications, Tools	9 Questions
Assignment 2	NumPy, Pandas, Matplotlib, Sklearn	12 Questions
Assignment 3	ML Activities, Data, Preprocessing, Evaluation	10 Questions

ASSIGNMENT 1 — Introduction to Machine Learning

Q1. Describe the basic concept of Machine Learning.

Machine Learning (ML) is a branch of Artificial Intelligence (AI) that enables computers to learn from data and improve their performance on tasks without being explicitly programmed.

Definition: ML is the science of making computers learn and act like humans by improving their learning over time in autonomous fashion, by feeding them data and information in the form of observations and real-world interactions.

Key Concepts:

- **Data:** Raw facts and figures that ML models learn from.
- **Model:** A mathematical function that maps input data to output predictions.
- **Training:** Process of feeding data to the model so it can learn patterns.
- **Testing:** Evaluating the model on unseen data to check its performance.
- **Features:** Input variables/attributes used to make predictions.
- **Label/Target:** The output variable the model is trying to predict.
- **Algorithm:** The procedure/method used to train the model (e.g., Decision Tree, SVM).

Working of ML (Simple Steps):

1. Collect Data
2. Prepare/Pre-process Data
3. Choose an Algorithm
4. Train the Model
5. Evaluate the Model
6. Deploy & Predict

Q2. Explain Applications of Machine Learning.

Application Area	Description & Example
Healthcare	Disease diagnosis, medical image analysis, drug discovery, patient monitoring. Example: Cancer detection using image recognition.
Finance	Fraud detection, credit scoring, stock market prediction, algorithmic trading. Example: Banks use ML to detect fraudulent transactions.
E-Commerce	Product recommendations, customer segmentation, dynamic pricing. Example: Amazon recommends products based on purchase history.
Natural Language Processing	Spam detection, sentiment analysis, language translation, chatbots. Example: Gmail spam filter uses ML.
Computer Vision	Face recognition, object detection, autonomous vehicles. Example: Tesla self-driving cars use ML algorithms.
Agriculture	Crop disease detection, yield prediction, soil analysis. Example: Drones use ML to detect infected crops.
Entertainment	Music/movie recommendations, content filtering. Example: Netflix uses ML for personalized movie suggestions.
Cybersecurity	Intrusion detection, malware analysis, network anomaly detection.

Q3. List out Types of Machine Learning and Explain any one in detail.

Types of Machine Learning:

- 1. Supervised Learning
- 2. Unsupervised Learning
- 3. Reinforcement Learning
- 4. Semi-Supervised Learning

Supervised Learning (in detail):

In supervised learning, the algorithm is trained on a **labeled dataset** — meaning each input has a corresponding correct output. The model learns to map input to output by minimizing the error between its predictions and actual labels.

Key Characteristics:

- Uses labeled training data (input-output pairs)
- Goal: Learn a function $f(x) \rightarrow y$
- Feedback is available (known correct answers)
- Used for prediction and classification tasks

Two categories:

- **Classification:** Output is a category. Example: Email spam/not spam, disease positive/negative.
- **Regression:** Output is a continuous value. Example: House price prediction, temperature forecasting.

Popular Supervised Learning Algorithms: Linear Regression, Logistic Regression, Decision Tree, Random Forest, SVM, K-Nearest Neighbors (KNN), Neural Networks.

Example: Training a model to recognize handwritten digits. Input = pixel values of digit image, Output = digit (0–9). The model learns from thousands of labeled images.

Q4 & Q5. Difference between Supervised and Unsupervised Machine Learning.

Supervised Learning	Unsupervised Learning
Uses labeled data	Uses unlabeled data
Output/target variable is known	No predefined output variable
Learns input-output mapping	Finds hidden patterns in data
Goal: Predict output for new data	Goal: Discover structure in data
Example: Email classification	Example: Customer segmentation
Algorithms: SVM, Decision Tree	Algorithms: K-Means, DBSCAN
Easier to evaluate (known labels)	Hard to evaluate (no ground truth)
Applications: Fraud detection, Diagnosis	Applications: Market basket analysis

Q6. Explain Reinforcement Learning in detail.

Reinforcement Learning (RL) is a type of ML where an **agent** learns to make decisions by interacting with an **environment** to maximize a **cumulative reward**.

Key Components of RL:

Agent	The learner/decision maker (e.g., a robot, game player)

Environment	The world with which agent interacts
State (S)	Current situation of the agent
Action (A)	Choices available to the agent
Reward (R)	Feedback signal — positive for good action, negative for bad
Policy (π)	Strategy that maps states to actions
Value Function	Expected cumulative reward from a state

How RL Works (Cycle): Agent observes State → Takes Action → Receives Reward from Environment → Updates policy → Repeat until optimal policy is learned.

Types of RL:

- **Model-Free RL:** Agent learns directly from interaction (Q-Learning, SARSA).
- **Model-Based RL:** Agent builds model of environment and plans ahead.

Applications: Game playing (AlphaGo, Chess), Robotics, Self-driving cars, Recommendation systems, Resource management.

Q7. Differences between Machine Learning and Human Learning.

Machine Learning	Human Learning
Requires large volumes of data	Can learn from very few examples
No emotions or intuition	Uses emotions, intuition & experience
Cannot generalize beyond training	Can generalize and transfer knowledge
Consistent, no fatigue	Affected by mood, tiredness
Processes data very fast	Slower but more adaptive
Cannot handle ambiguity well	Handles ambiguity naturally
No common sense reasoning	Has common sense
Requires explicit feature engineering	Automatically extracts relevant features
Needs retraining to adapt to change	Continuously adapts in real time
Cannot understand context deeply	Understands context and nuance

Q8. Provide an example for each type of ML (Supervised, Unsupervised, Reinforcement).

Type	Application	How it Works
Supervised Learning	Email Spam Detection	Model trained on labeled emails (spam/not spam). Learns features
Unsupervised Learning	Customer Segmentation	Groups customers by purchase behavior without predefined catego
Reinforcement Learning	Playing Chess / AlphaGo	Agent (program) plays games, receives reward for good moves, pe

Q9. List all popular tools/technologies used in Machine Learning and explain them.

Tool/Technology	Description

Python	Most popular programming language for ML. Simple syntax, rich ML libraries, large community support.
NumPy	Library for numerical computing. Provides support for large multi-dimensional arrays and matrices.
Pandas	Data manipulation and analysis library. Provides DataFrame structure for handling structured data.
Matplotlib/Seaborn	Data visualization libraries. Used to plot graphs, charts, heatmaps for data analysis.
Scikit-learn (sklearn)	ML library providing simple tools for classification, regression, clustering, preprocessing.
TensorFlow	Open-source deep learning framework by Google. Used for building and training neural networks.
Keras	High-level neural network API built on top of TensorFlow. Easier to use for building DL models.
PyTorch	Deep learning framework by Facebook. Popular in research for its dynamic computation graphs.
Jupyter Notebook	Interactive web-based environment to write and run code, visualize data, document work.
R Language	Statistical computing language. Used for data analysis, statistical modeling, visualization.
Apache Spark (MLlib)	Distributed ML library for big data. Handles large-scale data processing.
OpenCV	Computer vision library. Used for image/video processing, object detection, face recognition.

ASSIGNMENT 2 — NumPy, Pandas, Matplotlib & Sklearn

Q1. How to create NumPy Array & how to access it — explain with example.

What is NumPy? NumPy (Numerical Python) is a Python library for scientific computing. It provides a powerful N-dimensional array object called **ndarray**.

Creating NumPy Arrays:

From Python List:

```
import numpy as np arr = np.array([1, 2, 3, 4, 5]) # 1D array print(arr) # Output:
[1 2 3 4 5]
```

2D Array:

```
arr2d = np.array([[1,2,3],[4,5,6]]) print(arr2d) # Output: [[1 2 3] # [4 5 6]]
```

Using np.zeros():

```
arr = np.zeros((3,3)) # 3x3 array of zeros
```

Using np.ones():

```
arr = np.ones((2,4)) # 2x4 array of ones
```

Using np.arange():

```
arr = np.arange(1, 10, 2) # [1, 3, 5, 7, 9]
```

Using np.linspace():

```
arr = np.linspace(0, 1, 5) # [0, 0.25, 0.5, 0.75, 1.0]
```

Accessing NumPy Arrays:

Indexing:

```
arr = np.array([10, 20, 30, 40]) print(arr[0]) # 10 (first element) print(arr[-1])
# 40 (last element)
```

Slicing:

```
print(arr[1:3]) # [20, 30] print(arr[:2]) # [10, 20]
```

2D Indexing:

```
arr2d = np.array([[1,2,3],[4,5,6]]) print(arr2d[0,1]) # 2 (row 0, col 1)
print(arr2d[1,2]) # 6
```

Boolean Indexing:

```
arr = np.array([5, 10, 15, 20]) print(arr[arr > 10]) # [15, 20]
```

Q2. List out NumPy Statistics Functions & explain any two in detail.

NumPy Statistical Functions:

- np.mean()
- np.median()
- np.std()
- np.var()
- np.min()
- np.max()
- np.sum()
- np.cumsum()
- np.percentile()
- np.corrcoef()

- np.histogram()

Explanation 1: np.mean() — Arithmetic Mean

Calculates the average of array elements. It sums all elements and divides by count.

```
arr = np.array([2, 4, 6, 8, 10]) print(np.mean(arr)) # Output: 6.0
print(np.mean(arr, axis=0)) # column-wise mean for 2D
```

Explanation 2: np.std() — Standard Deviation

Measures the spread/dispersion of data from the mean. Low std = data is clustered near mean. High std = data is spread out.

```
arr = np.array([2, 4, 4, 4, 5, 5, 7, 9]) print(np.std(arr)) # Output: 2.0
print(np.var(arr)) # Variance = std^2 = 4.0
```

Q3. Explain Pandas Series with example.

What is Pandas Series? A Pandas Series is a **one-dimensional labeled array** that can hold data of any type (integers, strings, floats, etc.). It is like a column in a spreadsheet.

Creating a Series:

```
import pandas as pd # From a list s = pd.Series([10, 20, 30, 40]) print(s) # Output:
# 0 10 # 1 20 # 2 30 # 3 40 # dtype: int64
```

Series with Custom Index:

```
marks = pd.Series([85, 90, 78], index=['Maths','Science','English'])
print(marks['Maths']) # 85 print(marks[1:3]) # Science and English
```

Key Properties of Series:

- s.values — returns the underlying NumPy array
- s.index — returns the index labels
- s.dtype — data type of elements
- s.shape — shape of the series
- s.describe() — summary statistics

Q4. Explain Pandas Plotting with example.

Pandas Plotting: Pandas has built-in plotting capabilities using Matplotlib in the backend. You can call .plot() on a Series or DataFrame.

Plot Type	Syntax	Use Case
Line Plot	`df.plot(kind='line') – default`	Trends over time
Bar Chart	`df.plot(kind='bar')`	Category comparison
Histogram	`df.plot(kind='hist')`	Frequency distribution
Scatter Plot	`df.plot(kind='scatter', x='A', y='B')`	Correlation between variables
Box Plot	`df.plot(kind='box')`	Statistical distribution
Area Plot	`df.plot(kind='area')`	Cumulative data
Pie Chart	`df.plot(kind='pie')`	Proportion/percentage

Example — Bar Chart:

```
import pandas as pd import matplotlib.pyplot as plt data = {'Subject':
['Maths','Science','English'], 'Marks': [85,90,78]} df = pd.DataFrame(data)
df.plot(kind='bar', x='Subject', y='Marks', color='steelblue', title='Student
```

```
Marks') plt.tight_layout() plt.show()
```

Q5. Explain Pie Chart of Matplotlib.

A **Pie Chart** is a circular chart divided into slices to show the proportion of each category relative to the total. Each slice represents a percentage/fraction.

Syntax:

```
matplotlib.pyplot.pie(x, labels=None, explode=None, autopct=None, colors=None,
shadow=False, startangle=0)
```

Parameters:

- **x:** Array of values for each slice
- **labels:** List of strings as labels for each slice
- **explode:** Fraction to offset each slice from center
- **autopct:** String format for percentage labels, e.g. '%1.1f%%'
- **colors:** List of colors for each slice
- **shadow:** Boolean — adds shadow effect
- **startangle:** Rotation angle of chart (degrees)

Complete Example:

```
import matplotlib.pyplot as plt labels = ['Python','Java','C++','JavaScript'] sizes
= [40, 25, 20, 15] explode = (0.1, 0, 0, 0) # Explode Python slice colors =
['#ff9999','#66b3ff','#99ff99','#ffcc99'] plt.pie(sizes, labels=labels,
explode=explode, autopct='%1.1f%%', colors=colors, shadow=True, startangle=90)
plt.title('Programming Languages Usage') plt.axis('equal') # Equal aspect ratio
plt.show()
```

Q6. Explain Key Concepts and Features of Sklearn (Scikit-learn).

What is Sklearn? Scikit-learn is a free, open-source Machine Learning library for Python. It provides simple and efficient tools for data mining and data analysis, built on NumPy, SciPy, and Matplotlib.

Key Features of Scikit-learn:

Feature	Description
Supervised Learning Algorithms	Classification (SVM, Decision Tree, KNN, Logistic Regression) and Regression (Linear Regression, Ridge, Lasso)
Unsupervised Learning	Clustering (K-Means, DBSCAN), Dimensionality Reduction (PCA, t-SNE)
Model Selection & Evaluation	Cross-validation, GridSearchCV, train_test_split, performance metrics
Preprocessing	StandardScaler, MinMaxScaler, LabelEncoder, OneHotEncoder, imputation
Feature Selection	SelectKBest, RFE (Recursive Feature Elimination), feature importance
Pipeline	Chain multiple steps (preprocessing + model) into a single workflow
Ensemble Methods	RandomForest, GradientBoosting, AdaBoost, Bagging

Basic Sklearn Workflow:

```
from sklearn.model_selection import train_test_split from sklearn.preprocessing
import StandardScaler from sklearn.svm import SVC from sklearn.metrics import
accuracy_score X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2) scaler = StandardScaler() X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test) model = SVC() model.fit(X_train, y_train) y_pred
= model.predict(X_test) print(accuracy_score(y_test, y_pred))
```

Q7. Write a NumPy program to convert Python list to NumPy array.

```
import numpy as np # 1D List to NumPy array python_list = [1, 2, 3, 4, 5] np_array = np.array(python_list) print('Python List:', python_list) print('NumPy Array:', np_array) print('Type:', type(np_array)) # # 2D List to NumPy array matrix_list = [[1,2,3],[4,5,6],[7,8,9]] np_matrix = np.array(matrix_list) print('\n2D NumPy Array:') print(np_matrix) print('Shape:', np_matrix.shape) # (3, 3) print('Dimensions:', np_matrix.ndim) # 2
```

Output:

```
Python List: [1, 2, 3, 4, 5] NumPy Array: [1 2 3 4 5] Type: 2D NumPy Array: [[1 2 3] [4 5 6] [7 8 9]] Shape: (3, 3) Dimensions: 2
```

Q8. Write a NumPy program to split an array of 14 elements into 3 arrays (2, 4, 8 elements).

```
import numpy as np # Create array of 14 elements arr = np.arange(1, 15) # [1, 2, 3, ..., 14] print('Original Array:', arr) # Split into 3 parts: 2, 4, and 8 elements # np.split(array, indices) — split at indices 2 and 6 result = np.split(arr, [2, 6]) # index 2 and 2+4=6 print('\nAfter splitting:') print('Part 1 (2 elements):', result[0]) # [1, 2] print('Part 2 (4 elements):', result[1]) # [3, 4, 5, 6] print('Part 3 (8 elements):', result[2]) # [7, 8, 9, 10, 11, 12, 13, 14]
```

Output:

```
Original Array: [ 1 2 3 4 5 6 7 8 9 10 11 12 13 14] After splitting: Part 1 (2 elements): [1 2] Part 2 (4 elements): [3 4 5 6] Part 3 (8 elements): [ 7 8 9 10 11 12 13 14]
```

Q9. Write a NumPy program to find maximum and minimum value of a 1D array.

```
import numpy as np # Create a 1D array arr = np.array([45, 12, 78, 34, 56, 90, 23, 67]) print('Array:', arr) # Find maximum value max_val = np.max(arr) print('Maximum Value:', max_val) # 90 # Find minimum value min_val = np.min(arr) print('Minimum Value:', min_val) # 12 # Index of max and min print('Index of Max:', np.argmax(arr)) # 5 print('Index of Min:', np.argmin(arr)) # 1 # Range (max - min) print('Range:', max_val - min_val) # 78
```

Output:

```
Array: [45 12 78 34 56 90 23 67] Maximum Value: 90 Minimum Value: 12 Index of Max: 5 Index of Min: 1 Range: 78
```

Q10. Explain syntax for: (1) Remove Duplicate Rows, (2) Clean Empty Cells in Pandas DataFrame.

1) Remove Duplicate Rows — drop_duplicates()

```
import pandas as pd df = pd.DataFrame({'Name': ['Alice','Bob','Alice','Charlie'], 'Age': [25, 30, 25, 35]}) # Remove all duplicate rows df_clean = df.drop_duplicates() print(df_clean) # Remove duplicates based on specific column df_clean2 = df.drop_duplicates(subset=['Name'], keep='first') # keep='first' — keep first occurrence (default) # keep='last' — keep last occurrence # keep=False — drop all duplicates
```

2) Clean Empty Cells (NaN Values) — dropna() and fillna()

```
import pandas as pd import numpy as np df = pd.DataFrame({'Name':['Alice',np.nan,'Bob'], 'Age':[25, 30, np.nan]}) # Method 1: Drop rows with any NaN df1 = df.dropna() # Drop rows where all values are NaN df2 = df.dropna(how='all') # Drop only if NaN in specific column df3 = df.dropna(subset=['Name']) # Method 2: Fill NaN with a value df4 = df.fillna(0) # fill with 0 df5 = df['Age'].fillna(df['Age'].mean()) # fill with mean # Method 3:
```

```
Forward fill / Backward fill df6 = df.fillna(method='ffill') # forward fill df7 =
df.fillna(method='bfill') # backward fill
```

Q11. What are the steps to create a Pie Chart and Bar Graph using Matplotlib?

Steps to Create a Pie Chart:

Step 1: Import matplotlib.pyplot as plt

Step 2: Prepare data — list of sizes/values and labels

Step 3: Call plt.pie(sizes, labels=labels, autopct='%1.1f%%')

Step 4: Add title with plt.title()

Step 5: Call plt.axis('equal') for perfect circle

Step 6: Display with plt.show()

```
import matplotlib.pyplot as plt labels = ['A','B','C','D'] sizes = [30, 25, 25, 20]
plt.pie(sizes, labels=labels, autopct='%1.1f%%', startangle=90) plt.title('Pie
Chart Example') plt.axis('equal') plt.show()
```

Steps to Create a Bar Graph:

Step 1: Import matplotlib.pyplot as plt

Step 2: Prepare categories (x) and values (y)

Step 3: Call plt.bar(x, y) for vertical or plt.barh(x, y) for horizontal

Step 4: Add labels: plt.xlabel(), plt.ylabel()

Step 5: Add title with plt.title()

Step 6: Display with plt.show()

```
import matplotlib.pyplot as plt categories = ['Math','Science','English','History']
marks = [85, 90, 78, 88] plt.bar(categories, marks,
color=['blue','green','red','orange']) plt.xlabel('Subjects') plt.ylabel('Marks')
plt.title('Student Marks - Bar Graph') plt.ylim(0, 100) for i, v in
enumerate(marks): plt.text(i, v+1, str(v), ha='center') # Add value labels
plt.show()
```

Q12. Discuss the steps involved in reading a CSV file in Pandas.

What is CSV? CSV (Comma Separated Values) is a plain text file format where data is stored in rows, with values separated by commas.

Steps to Read CSV file in Pandas:

Step 1: Import Pandas:

```
import pandas as pd
```

Step 2: Read the CSV file:

```
df = pd.read_csv('filename.csv')
```

Step 3: View first few rows:

```
print(df.head()) # First 5 rows print(df.tail()) # Last 5 rows
```

Step 4: Check shape & info:

```
print(df.shape) # (rows, columns) print(df.info()) # Data types, non-null counts
print(df.describe()) # Statistical summary
```

Step 5: Access columns:

```
print(df['Name']) # Single column print(df[['Name','Age']]) # Multiple columns
```

Step 6: Handle missing values:

```
print(df.isnull().sum()) # Count NaN values df.fillna(0, inplace=True) # Replace
NaN
```

Common pd.read_csv() Parameters:

- **sep:** Delimiter (default ',')
- **header:** Row to use as column names (default 0)
- **index_col:** Column to use as row index
- **usecols:** List of columns to read
- **nrows:** Number of rows to read
- **skiprows:** Rows to skip from beginning
- **encoding:** File encoding (e.g., 'utf-8')
- **na_values:** Values to treat as NaN

ASSIGNMENT 3 — Data, Preprocessing & Model Evaluation

Q1. Describe different types of Machine Learning Activities.

Machine Learning activities refer to the different types of tasks/problems that ML algorithms can solve.

Activity	Description	Example
Classification	Assigns input to one of several predefined categories	Email: spam/not spam, Disease: yes/no
Regression	Predicts a continuous numerical output value	House price prediction, temperature
Clustering	Groups unlabeled data into similar clusters	Customer segmentation, document grouping
Dimensionality Reduction	Reduces number of features preserving info	PCA for image compression
Anomaly Detection	Identifies unusual/outlier data points	Fraud detection, fault detection
Recommendation	Suggests items based on past behavior	Netflix movies, Amazon products
Natural Language Processing	Understands & generates human language	Chatbots, translation, sentiment
Computer Vision	Interprets visual information from images	Face recognition, object detection
Association Rule Learning	Finds relationships between variables	Market basket: milk→bread→butter
Reinforcement Learning	Agent learns from actions and rewards	Game playing, robotics control

Q2. Explain Types of Data in Machine Learning.

Data is the most fundamental component of Machine Learning. Understanding data types is essential for proper preprocessing and model selection.

A) Based on Structure:

- **Structured Data:** Organized in rows and columns (tabular). Easily stored in databases. Example: CSV files, Excel sheets, SQL tables
- **Unstructured Data:** No predefined format or organization. Example: Images, audio files, text documents, videos
- **Semi-Structured Data:** Has some organization but not in tabular form. Example: JSON, XML, HTML files, emails

B) Based on Data Types (Statistical):

Data Type	Description	Example	ML Use
Nominal (Categorical)	Categories with no order	Colors, Gender, Country	One-Hot Encoding
Ordinal	Categories with meaningful order	Ratings (1-5), Grades A>B>C	Label Encoding
Discrete (Numerical)	Countable integer values	No. of students, clicks	Direct use
Continuous (Numerical)	Any value in a range	Height, Weight, Temperature	Normalization/Scaling
Binary	Only two values	Yes/No, 0/1, True/False	Direct / Encoding
Text/String	Sequence of characters	Names, Sentences, Reviews	NLP Techniques

Date/Time	Time-based data	Timestamps, Dates	Feature extraction
Image	Pixel data (2D/3D array)	Photos, Medical scans	CNN / Feature extraction

Q3. Explain Data Pre-processing.

Data Pre-processing is the process of transforming raw, messy, and incomplete data into a clean and usable format before feeding it to a ML model. Real-world data is often noisy, inconsistent, and incomplete — preprocessing improves model accuracy.

Steps in Data Pre-processing:

1. **Data Collection:** Gathering raw data from various sources — databases, APIs, CSV files, web scraping.
2. **Data Cleaning:** Handle missing values (drop/fill), remove duplicates, fix inconsistencies, correct errors.
3. **Data Integration:** Combining data from multiple sources into a unified dataset.
4. **Data Transformation:** Normalize/standardize numerical features, encode categorical variables.
5. **Data Reduction:** Reduce volume: dimensionality reduction (PCA), feature selection, sampling.
6. **Data Splitting:** Split into Training set (70-80%) and Testing set (20-30%).
7. **Feature Engineering:** Create new meaningful features from existing ones to improve model.

Encoding Techniques:

- **Label Encoding:** Convert categories to numbers. Male=0, Female=1
- **One-Hot Encoding:** Creates binary column for each category (avoids ordinal assumption)

Scaling Techniques:

- **Normalization (Min-Max Scaling):** Rescales data to [0,1]. Formula: x' = (x - min)/(max - min)
- **Standardization (Z-score):** Rescales to mean=0, std=1. Formula: x' = (x - mean)/std

Q4. Explain Data Quality and Remediation.

Data Quality refers to the degree to which data is accurate, complete, consistent, timely, and reliable for its intended use. Poor data quality leads to poor ML model performance ('Garbage In, Garbage Out').

Dimensions of Data Quality:

Quality Dimension	Description	Example Issue
Accuracy	Data correctly represents real-world values	Age = -5 is inaccurate
Completeness	No missing values	Customer record without phone number
Consistency	Same data across multiple sources	DOB differs in two tables
Timeliness	Data is up-to-date	Using 10-year-old sales data
Validity	Data follows defined rules/format	Email without '@' symbol
Uniqueness	No duplicate records	Same customer entered twice
Relevance	Data is useful for the task	Including irrelevant columns

Data Remediation Techniques:

- **Missing Data:** Drop rows/columns with many missing values, fill with mean/median/mode, use predictive imputation
- **Duplicate Data:** Use drop_duplicates() in Pandas, define unique keys
- **Inconsistent Data:** Standardize formats (date, phone, names), resolve conflicts
- **Noisy Data:** Apply smoothing techniques (binning, regression), remove outliers

- **Incorrect Data:** Validate against business rules, correct using domain knowledge
- **Imbalanced Data:** Use oversampling (SMOTE), undersampling, class weights

Q5. Give Difference between Categorical Data & Numeric Data.

Categorical Data	Numeric Data
Represents groups/categories	Represents measurable quantities
Non-numerical (text-based) labels	Numerical values (integers or floats)
Examples: Gender, Color, City	Examples: Age, Height, Salary, Temperature
Two types: Nominal and Ordinal	Two types: Discrete and Continuous
Cannot perform arithmetic operations	Can perform arithmetic (sum, mean, etc.)
Need encoding before ML (Label/One-Hot)	Can be used directly or scaled
Visualized with bar/pie charts	Visualized with histogram/box plots
Central tendency: Mode	Central tendency: Mean, Median
Low cardinality (few unique values)	High cardinality (many unique values)
Stored as strings/objects in Pandas	Stored as int64/float64 in Pandas

Q6. Explain Process of Machine Learning with Diagram.

The Machine Learning process is a systematic workflow from raw data to a deployed predictive model.

#	Step	Description
1	**Data Collection**	Gather raw data from databases, APIs, files, web scraping, sensors
2	**Data Pre-processing**	Clean, transform, encode, normalize data — remove noise & missing values
3	**Exploratory Data Analysis**	Visualize, understand patterns, check correlations and distributions
4	**Feature Engineering**	Select and create meaningful features that improve model performance
5	**Model Selection**	Choose appropriate ML algorithm (SVM, Decision Tree, etc.)
6	**Model Training**	Feed training data to algorithm, model learns patterns from data
7	**Model Evaluation**	Test on unseen data, check accuracy, precision, recall, F1 score
8	**Hyperparameter Tuning**	Optimize model settings using GridSearchCV, cross-validation
9	**Model Deployment**	Integrate model into production system / web application / API
10	**Monitoring & Update**	Monitor performance, retrain with new data periodically

Q7. Explain K-Fold Cross Validation method for data partitioning.

What is K-Fold Cross Validation? K-Fold Cross Validation is a model evaluation technique where the dataset is divided into **K equal folds (subsets)**. The model is trained K times, each time using K-1 folds for training and 1 fold for testing.

Steps in K-Fold Cross Validation (Example: K=5):

- Divide the dataset into K=5 equal folds
- Iteration 1: Use Fold 1 as test, Folds 2-5 as training → Calculate accuracy
- Iteration 2: Use Fold 2 as test, Folds 1,3,4,5 as training → Calculate accuracy
- Iteration 3: Use Fold 3 as test, others as training → Calculate accuracy
- Iteration 4: Use Fold 4 as test, others as training → Calculate accuracy
- Iteration 5: Use Fold 5 as test, Folds 1-4 as training → Calculate accuracy
- Final Score = Average of all 5 accuracy scores

Iteration	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Accuracy
1	TEST	TRAIN	TRAIN	TRAIN	TRAIN	A1
2	TRAIN	TEST	TRAIN	TRAIN	TRAIN	A2
3	TRAIN	TRAIN	TEST	TRAIN	TRAIN	A3
4	TRAIN	TRAIN	TRAIN	TEST	TRAIN	A4
5	TRAIN	TRAIN	TRAIN	TRAIN	TEST	A5
Final Score						**Mean(A1..A5)**

Advantages of K-Fold CV: Better utilization of data, reduces variance in evaluation, more reliable than single train-test split, every data point used for both training and testing.

Common values of K: K=5 or K=10 are most widely used. K=n is called Leave-One-Out Cross Validation (LOOCV).

Q8. Explain Performance Evaluation Methods: (1) Cross Validation, (2) Confusion Matrix.

1) Cross Validation:

Cross Validation is a statistical technique to evaluate how well a model generalizes to independent datasets. It prevents overfitting and gives a reliable estimate of model performance.

Types of Cross Validation:

- **K-Fold CV:** Dataset split into K folds; model trained K times (explained in Q7)
- **Stratified K-Fold:** Ensures each fold has same class distribution as original dataset
- **Leave-One-Out (LOOCV):** K = n (each sample is test once); computationally expensive
- **Holdout Validation:** Simple train/test split; fast but may have high variance
- **Time Series CV:** For sequential data; uses past to predict future; avoids data leakage

```
from sklearn.model_selection import cross_val_score from sklearn.tree import
DecisionTreeClassifier model = DecisionTreeClassifier() scores =
cross_val_score(model, X, y, cv=5) # 5-Fold CV print('CV Scores:', scores)
print('Mean Accuracy:', scores.mean())
```

2) Confusion Matrix:

A Confusion Matrix is a table that shows the performance of a classification model. It compares **actual vs predicted** values.

	PREDICTED	
	Positive (1)	**Negative (0)**

ACTUAL	Positive (1)	TP (True Positive)	FN (False Negative)
	Negative (0)	FP (False Positive)	TN (True Negative)

Terms Explained:

- **TP (True Positive):** Actual=Positive, Predicted=Positive — CORRECT prediction
- **TN (True Negative):** Actual=Negative, Predicted=Negative — CORRECT prediction
- **FP (False Positive):** Actual=Negative, Predicted=Positive — WRONG (Type I Error)
- **FN (False Negative):** Actual=Positive, Predicted=Negative — WRONG (Type II Error)

Metrics derived from Confusion Matrix:

Metric	Formula	Meaning
Accuracy	`(TP + TN) / (TP+TN+FP+FN)`	Overall correct predictions
Precision	`TP / (TP + FP)`	Of all predicted positive, how many are truly positive
Recall (Sensitivity)	`TP / (TP + FN)`	Of all actual positive, how many were correctly identified
F1 Score	`2 * (Precision * Recall) / (Precision + Recall)`	Harmonic mean of Precision and Recall
Specificity	`TN / (TN + FP)`	True Negative Rate

Q9. Explain Outliers & How to Handle Outliers.

What is an Outlier? An outlier is a data point that is **significantly different** from other observations. It lies far from the mean/median and can distort statistical analysis and reduce ML model accuracy.

Example: Ages of students: [18, 19, 20, 21, 22, 85] — Here 85 is an outlier.

Causes of Outliers:

- Measurement errors or data entry mistakes
- Experimental errors
- Natural variability in data
- Intentional outliers (fraud)
- Data from different population

Types of Outliers:

- **Point Outlier:** Single data point far from others
- **Contextual Outlier:** Outlier in specific context (e.g., temperature 40°C in winter)
- **Collective Outlier:** Group of points are outliers together

Methods to Detect Outliers:

- **Z-Score:** Values with |Z| > 3 are outliers. Z = (x - mean)/std
- **IQR Method:** Outlier if < Q1 - 1.5*IQR or > Q3 + 1.5*IQR
- **Box Plot:** Visual method — points beyond whiskers are outliers
- **Scatter Plot:** Visually identify extreme data points
- **Isolation Forest:** ML-based algorithm for anomaly/outlier detection

How to Handle Outliers:

```
import numpy as np import pandas as pd arr = [10, 12, 11, 14, 200, 13, 9, 11] df = pd.DataFrame({'Value': arr}) # Method 1: Z-Score removal from scipy import stats z = np.abs(stats.zscore(df['Value'])) df_clean = df[z < 3] # Method 2: IQR Method Q1 = df['Value'].quantile(0.25) Q3 = df['Value'].quantile(0.75) IQR = Q3 - Q1 df_clean2 = df[(df['Value'] >= Q1 - 1.5*IQR) & (df['Value'] <= Q3 + 1.5*IQR)] #
```

```
Method 3: Capping (Winsorization) df['Value'] = df['Value'].clip(lower=Q1-1.5*IQR, upper=Q3+1.5*IQR)
```

Method	Description
Remove	Delete outlier rows (if due to error and few in number)
Cap/Floor	Replace with maximum/minimum acceptable value (Winsorization)
Transform	Apply log/sqrt transformation to reduce outlier effect
Impute	Replace with mean/median/mode like missing values
Keep	If genuine extreme value, keep and use robust algorithms
Separate Model	Build separate model for outlier-prone segments

Q10. Describe the purpose of Dimensionality Reduction technique in data preprocessing.

What is Dimensionality Reduction? Dimensionality reduction is the process of **reducing the number of features (variables/dimensions)** in a dataset while retaining the most important information. It transforms data from a high-dimensional space to a lower-dimensional space.

Why is it needed? (Curse of Dimensionality)

- As features increase, model becomes complex and may overfit
- More dimensions = more computation time and memory
- Many features may be redundant or irrelevant
- Hard to visualize data with more than 3 dimensions
- Performance of distance-based algorithms degrades with high dimensions

Purposes/Benefits of Dimensionality Reduction:

Purpose	How it Helps
Reduces Overfitting	Fewer features means model generalizes better to new data
Improves Performance	Less computation → faster training and prediction
Removes Multicollinearity	Eliminates highly correlated/redundant features
Data Visualization	Reduce to 2D/3D to visualize clusters and patterns
Noise Reduction	Low-variance features often contain noise; removing them improves accuracy
Memory Efficiency	Less data to store and process
Better Interpretability	Fewer features make model easier to understand

Two Main Approaches:

- **Feature Selection:** Select the most relevant subset of original features. Methods: Filter (correlation), Wrapper (RFE), Embedded (Lasso, Decision Tree feature importance).
- **Feature Extraction:** Create new features by combining original ones. Methods: PCA, t-SNE, LDA, Autoencoders.

PCA (Principal Component Analysis) — Most Popular Technique:

PCA transforms the original features into a new set of orthogonal components (Principal Components) ordered by the amount of variance they explain. The first few PCs capture most of the data's variance.

```
from sklearn.decomposition import PCA from sklearn.preprocessing import StandardScaler # Standardize first scaler = StandardScaler() X_scaled = scaler.fit_transform(X) # Apply PCA — reduce to 2 dimensions pca =
```

```
PCA(n_components=2) X_reduced = pca.fit_transform(X_scaled) print('Original
shape:', X_scaled.shape) # e.g., (1000, 50) print('Reduced shape:',
X_reduced.shape) # e.g., (1000, 2) print('Variance explained:',
pca.explained_variance_ratio_)
```

— End of All Assignments | Best of Luck for Your Exam! —

Government Polytechnic for Girls, Surat | Introduction to Machine Learning | Sem IV